Litigant Portal

Technical Architecture & Implementation Plan

Version: 1.0 **Date:** October 23, 2025 **Status:** Planning Phase **Organization:** Free Law Project

---

Executive Summary

The Litigant Portal is a comprehensive access-to-justice platform designed to assist self-represented litigants in navigating the court system. This document outlines the technical architecture, recommended technology stack, implementation strategy, and scalability considerations for building a mobile-first, accessible, and open-source solution.

Key Architectural Decisions

- **Technology Stack:** Python/Django with Django Cotton, HTMX, and Tailwind CSS
- **Cloud Infrastructure:** Amazon Web Services (AWS)
- **Database Strategy:** PostgreSQL with repository pattern for data source abstraction
- **Authentication:** AWS Cognito with multi-factor authentication support
- **Accessibility:** WCAG 2.1 Level AA compliance (minimum)
- **Licensing:** GNU Affero General Public License v3.0 (AGPL-3.0)
- **Internationalization:** Plugin-based translation system for community contributions
- **Payment Processing:** Abstract payment interface supporting multiple providers

---

Project Overview

Mission

Empower self-represented litigants with technology-driven tools to navigate court systems effectively and independently, thereby reducing the justice gap while simultaneously easing administrative burdens on court staff.

Initial Deployment

Portsmouth General District Court (3rd Judicial District, Virginia)

Core Capabilities

Phase 1 - Minimum Viable Product (MVP):

- User authentication and profile management
- Document upload and secure storage
- Electronic filing (eFiling) integration
- Court form library access
- Basic case management dashboard
- Email and SMS notifications

Phase 2 - Enhanced Features:

- Guided interview workflows (A2J Author, HotDocs, DocAssemble integration)
- Payment processing for court fees
- Fee waiver application system
- Automated deadline reminders
- Multi-jurisdiction support

Phase 3 - AI Integration:

- Retrieval-Augmented Generation (RAG) for legal guidance
- Plain-language legal explanations
- Intelligent form selection assistance
- Document analysis and OCR capabilities

Critical Requirements

- Mobile-first responsive design
- Web Content Accessibility Guidelines (WCAG) 2.1 Level AA compliance
- Decoupled architecture supporting multiple data sources
- Open-source licensing promoting community contribution
- Scalable infrastructure supporting statewide and national expansion

---

Recommended Technology Stack

Frontend Architecture

Primary Technologies:

- **Django 5.0+** - Web framework
- **Django Cotton** - Component-based template system
- **HTMX** - Dynamic interactions without heavy JavaScript

- **Tailwind CSS** - Utility-first CSS framework

Rationale:

The recommended stack leverages the Free Law Project team's existing expertise in Django, Python, and Tailwind CSS while providing a robust foundation for mobile-first, accessible web applications.

Advantages:

- Server-rendered HTML improves initial page load performance and search engine optimization
- Enhanced accessibility through semantic HTML structure
- Progressive enhancement ensures functionality without JavaScript
- Single-language development (Python) reduces context switching
- Mature ecosystem with extensive security and accessibility tooling

Trade-offs:

- Less suitable for highly interactive single-page applications
- Limited code sharing if native mobile applications are required in the future
- Smaller developer community compared to React ecosystem

Backend Architecture

Primary Technologies:

- **Django REST Framework (DRF)** - API framework
- **Celery** - Asynchronous task processing
- **Redis** - Caching and message broker

Design Pattern:

- **Repository Pattern** - Abstraction layer for data access
- **Service Layer** - Business logic isolation
- **Adapter Pattern** - Multiple data source integration

Rationale:

The repository pattern provides critical flexibility for integrating with multiple court systems, external APIs (CourtListener), and future data sources without coupling business logic to specific implementations.

Database Infrastructure

Primary Database: PostgreSQL 15+ (AWS RDS Multi-AZ) **Cache Layer:** Redis (AWS ElastiCache) **Document Storage:** Amazon S3 with encryption and versioning

Rationale:

- PostgreSQL provides robust ACID compliance, excellent JSON support, and full-text search capabilities
- Redis enables high-performance caching, session management, and background job queuing
- S3 offers scalable, durable document storage with built-in encryption and lifecycle management

Authentication & Security

AWS Cognito User Pools

Features:

- Multi-factor authentication (MFA)
- Social identity provider integration (Google, Facebook)
- SAML/OIDC support for court staff single sign-on
- Built-in password policies and account recovery
- Automatic scaling

System Architecture

High-Level Architecture Diagram

```None
┌──────────────────────────────────────────────────────────────┐
│                     User Interface Layer                     │
│         (Django Cotton + HTMX + Tailwind CSS)                │
│                                                               |
│  - Mobile-responsive design                                  │
│  - WCAG 2.1 AA accessible components                         │
│  - Progressive enhancement                                   │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
```

```
│                   API Gateway Layer                        │
│            (Django REST Framework)                         │
│                                                             │
│  - Authentication & authorization (Cognito)                │
│  - Request validation & rate limiting                      │
│  - API versioning                                          │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│                   Service Layer                            │
│              (Business Logic)                              │
│                                                             │
│  - Form processing & validation                            │
│  - Document assembly workflows                             │
│  - eFiling coordination                                    │
│  - Notification management                                 │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│            Repository/Adapter Layer                        │
│         (Data Source Abstraction)                          │
│                                                             │
│  ┌──────────────┬──────────────┬──────────────────────┐   │
│  │ Court API    │ CourtListener│ Payment Processing   │   │
│  │ Adapters     │ Integration  │ Adapters             │   │
│  │              │              │                      │   │
│  │ - Portsmouth │ - Case law   │ - Stripe             │   │
│  │ - Virginia   │ - Dockets    │ - AWS Pay (future)   │   │
│  │ - Generic    │ - Opinions   │ - Configurable       │   │
│  └──────────────┴──────────────┴──────────────────────┘   │
└────────────────────────┬──────────────────────────────────┘
                         │
        ┌────────────────┼────────────────┐
        │                │                │
┌───────▼────────┐  ┌────▼──────────┐  ┌──▼──────────────┐
│ PostgreSQL     │  │ Court         │  │ External APIs   │
│ (Primary DB)   │  │ Systems       │  │                 │
│                │  │               │  │ - CourtListener│
│ - User data    │  │ - Case Mgmt   │  │ - Payment       │
│ - Cases        │  │ - eFiling     │  │   gateways      │
│ - Documents    │  │ - Dockets     │  │                 │
└────────────────┘  └───────────────┘  └─────────────────┘
```

Data Abstraction Architecture

The repository pattern enables seamless integration with multiple data sources:

```
None

┌──────────────────────────────────────────────────────────────┐
│                   Application Layer                          │
│              (Views, Forms, Templates)                       │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│                    Service Layer                             │
│                                                               │
│  class CaseService:                                          │
│      def __init__(self, repository: CaseRepository):         │
│          self.repository = repository                        │
│                                                               │
│      async def get_user_cases(self, user_id):                │
│          return await self.repository.get_user_cases(...)    │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│              Abstract Repository Interface                   │
│                  (CaseRepository)                            │
│                                                               │
│  - get(id) -> Case                                           │
│  - list(filters) -> List[Case]                               │
│  - create(case) -> Case                                      │
│  - update(id, case) -> Case                                  │
│  - get_by_case_number(number, jurisdiction) -> Case          │
└──────────────────────────┬───────────────────────────────────┘
                           │
          ┌────────────────┼────────────────┐
          │                │                │
┌─────────▼────────┐  ┌────▼──────────┐  ┌──────▼──────────────┐
│ PostgreSQL       │  │CourtListener│  │ Court System API   │
│ Adapter          │  │ Adapter      │  │ Adapter            │
│                  │  │              │  │                    │
│ - Local storage  │  │ - Read-only  │  │ - Portsmouth CMS   │
│ - Full CRUD      │  │ - Search     │  │ - eFiling          │
│ - Primary source│  │ - Enrichment│  │ - Court-specific   │
└──────────────────┘  └──────────────┘  └────────────────────┘
```

Benefits:

- Decoupled business logic from data sources
- Easy integration with different court systems
- Simplified testing through mock repositories
- Future-proof architecture supporting technology migration

AWS Infrastructure Architecture

Cloud Infrastructure Diagram

None

```
┌──────────────────────────────────────────────────────────────────┐
│                      Route 53 (DNS)                              │
└──────────────────────────┬───────────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────────┐
│                    CloudFront CDN                                │
│                                                                   |
│  - SSL/TLS termination                                           │
│  - Global edge caching                                           │
│  - DDoS protection (AWS Shield)                                  │
│  - Geographic content delivery                                   │
└──────────────────────────┬───────────────────────────────────────┘
                           │
         ┌─────────────────┴─────────────────┐
         │                                   |
┌────────▼─────────────┐            ┌────────▼──────────────────┐
│  S3 Bucket           |            |  Application Load         |
|  (Static Assets)     |            |  Balancer (ALB)           |
|                      |            |                           |
|  - CSS, JavaScript   |            |  - SSL/TLS termination    |
|  - Images, fonts     |            |  - Health checks          |
|  - Public content    |            |  - Path-based routing     |
└──────────────────────┘            └──────────┬────────────────┘
                                               |
                                               |
                             ┌─────────────────┴─────────────────┐
                             |                 |                 |
                     ┌───────▼─────────┐  ┌──▼──────┐  ┌──────▼──────┐
                     |  ECS Fargate    |  | Lambda  |  | Cognito     |
                     |  Service        |  |Functions|  |User Pool    |
                     |                 |  |         |  |             |
                     |  - Django app   |  |- Utility|  |- Auth       |
                     |  - Auto-scaling|  |  tasks  |  |- MFA        |
```

```
                  |  - Multi-AZ    |  |- Webhooks| |- Social  |
                  └────────┬────────┘  └──────────┘  └──────────┘
                           |
            ┌──────────────┼──────────────┐
            |              |              |
    ┌───────▼────────┐  ┌─▼──────┐  ┌──▼─────────────┐
    | RDS            |  |ElastiCache| | S3 Bucket      |
    | PostgreSQL     |  |  Redis    | | (Documents)    |
    |                |  |           | |                |
    | - Multi-AZ     |  | - Sessions| | - Encrypted    |
    | - Encrypted    |  | - Cache   | | - Versioned    |
    | - Automated    |  | - Celery  | | - Lifecycle    |
    |   backups      |  |   broker  | |   policies     |
    └────────────────┘  └───────────┘ └────────────────┘


┌──────────────────────────────────────────────────────────┐
|              Supporting AWS Services                      |
├──────────────────────────────────────────────────────────┤
|  - CloudWatch: Logging, metrics, monitoring, alarms       |
|  - Secrets Manager: API keys, database credentials        |
|  - Systems Manager: Configuration parameters              |
|  - SES: Transactional email delivery                      |
|  - SNS: SMS notifications                                 |
|  - SQS: Background job queuing                            |
|  - EventBridge: Scheduled task management                 |
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
|       Phase 3: AI/ML Services (Future Integration)        |
├──────────────────────────────────────────────────────────┤
|  - Bedrock: Foundation models for legal guidance          |
|  - OpenSearch: Vector database for RAG architecture       |
|  - Textract: Document OCR and data extraction             |
|  - Comprehend: Natural language processing                |
|  - SageMaker: Custom model training and deployment        |
└──────────────────────────────────────────────────────────┘
```

Infrastructure Characteristics

Scalability:

- ECS Fargate auto-scaling based on CPU utilization and request count
- RDS read replicas for improved read performance
- CloudFront edge caching reduces origin server load

- Elastic cache layer minimizes database queries

Reliability:

- Multi-AZ deployment for RDS and ECS services
- Automated database backups with 30-day retention
- Health checks and automatic container replacement
- CloudWatch alarms for proactive incident response

Security:

- Encryption at rest (RDS, S3) and in transit (TLS 1.2+)
- AWS Secrets Manager for credential management
- VPC isolation with private subnets for databases
- AWS WAF for application-layer attack protection

Internationalization Strategy

Plugin-Based Translation Architecture

To enable global community contributions, the portal implements a plugin-based translation system where language packs can be independently developed, reviewed, and maintained.

```
None
litigant-portal/
├── locale/                          # Core translations (bundled)
│   ├── en_US/
│   │   ├── LC_MESSAGES/
│   │   │   ├── django.po           # General UI strings
│   │   │   ├── forms.po            # Form labels
│   │   │   ├── legal.po            # Legal terminology
│   │   │   └── notifications.po    # Email/SMS templates
│   │   └── metadata.json
│   │
│   ├── es_MX/                       # Spanish (Mexico)
│   │   ├── LC_MESSAGES/
│   │   │   ├── django.po
│   │   │   ├── forms.po
│   │   │   ├── legal.po
│   │   │   └── notifications.po
│   │   └── metadata.json
```

```
│   │
│   └── README.md
│
├── locale_plugins/                  # Community contributions
│   ├── README.md
│   ├── CONTRIBUTING.md              # Contribution guidelines
│   │
│   ├── en_GB/                       # British English variant
│   │   ├── metadata.json
│   │   ├── LC_MESSAGES/
│   │   │   └── legal.po            # UK legal terms
│   │   └── CONTRIBUTORS.md
│   │
│   ├── fr_FR/                       # French
│   │   ├── metadata.json
│   │   ├── LC_MESSAGES/
│   │   └── CONTRIBUTORS.md
│   │
│   └── template/                    # Template for new languages
│       ├── metadata.json.template
│       └── HOWTO.md
```

Translation Metadata Structure

Each language plugin includes metadata ensuring quality and attribution:

```JSON
{
  "language_code": "es_MX",
  "language_name": "Español (México)",
  "language_name_english": "Spanish (Mexico)",
  "text_direction": "ltr",

  "maintainers": [
    {
      "name": "Community Contributor",
      "email": "contributor@example.org"
    }
  ],

  "legal_reviewer": {
    "name": "Legal Expert Name",
```

```
    "credentials": "Attorney credentials",
    "review_date": "2025-09-15"
  },

  "coverage": {
    "django": 98,
    "forms": 100,
    "legal": 95,
    "notifications": 100
  },

  "status": "production",
  "version": "1.2.0",
  "last_updated": "2025-10-15"
}
```

Quality Assurance

Requirements for Production Status:

- Minimum 95% translation coverage across all string categories
- Legal terminology review by qualified legal professional
- Accessibility testing with screen readers
- Regional appropriateness verification
- Active maintainer commitment

Payment Processing Architecture

Abstract Payment Interface

To support multiple payment processors and court-specific requirements, the portal implements an abstract payment processing interface:

```
None
┌──────────────────────────────────────────────────────────────┐
│                     Payment Service Layer                    │
│                   (Business Logic)                           │
│                                                               │
│  - Fee waiver validation                                     │
```

```
│  - Court fee calculation                                    │
│  - Payment workflow orchestration                           │
│  - Receipt generation                                       │
└─────────────────────────┬──────────────────────────────────┘
                          │
┌─────────────────────────▼──────────────────────────────────┐
│              Abstract Payment Processor Interface           │
│                                                              │
│  - create_payment_intent(amount, metadata)                  │
│  - confirm_payment(intent_id, method_id)                    │
│  - refund_payment(payment_id, amount, reason)               │
│  - get_payment_status(payment_id)                           │
│  - create_payment_method(type, details)                     │
└─────────────────────────┬──────────────────────────────────┘
                          │
         ┌────────────────┼────────────────┐
         │                │                │
┌────────▼────────┐  ┌────▼────────┐  ┌────▼──────────────┐
│ Stripe Adapter  │  │ AWS Payment│  │ Future Adapters   │
│                 │  │ Adapter    │  │                   │
│ - Credit cards  │  │ (Planned)  │  │ - Court-specific  │
│ - ACH transfers │  │            │  │ - Regional        │
│ - Fee waivers   │  │            │  │   processors      │
└─────────────────┘  └────────────┘  └───────────────────┘
```

Configuration-Driven Selection:

Different courts may require different payment processors based on existing contracts or regional requirements. The factory pattern enables jurisdiction-specific configuration:

```python
PAYMENT_PROCESSORS = {
    'default': {'type': 'stripe'},
    'portsmouth_va': {
        'type': 'stripe',
        'account_id': 'court_specific_account'
    }
}
```

Accessibility Compliance

WCAG 2.1 Level AA Standards

The portal is designed to meet or exceed Web Content Accessibility Guidelines (WCAG) 2.1 Level AA standards, ensuring equal access for users with disabilities.

Implementation Approach

1. Semantic HTML Structure

- Proper heading hierarchy (h1-h6)
- Semantic landmarks (header, nav, main, aside, footer)
- ARIA labels where necessary
- Form labels explicitly associated with inputs

2. Keyboard Navigation

- All functionality accessible via keyboard
- Logical tab order
- Visible focus indicators (3px outline, high contrast)
- Skip navigation links for screen reader users
- No keyboard traps

3. Color Contrast

- Minimum 4.5:1 contrast ratio for normal text
- Minimum 3:1 contrast ratio for large text (18pt+)
- Color not used as sole means of conveying information
- Tested color palette ensuring compliance

4. Touch Target Sizes

- Minimum 44x44 pixel interactive elements
- Adequate spacing between clickable elements
- Mobile-optimized form controls

5. Screen Reader Support

- Descriptive alt text for images
- ARIA live regions for dynamic content
- Proper table structure with headers
- Form error announcements
- Status messages communicated to assistive technologies

6. Responsive Design

- Text resizable to 200% without loss of functionality
- No horizontal scrolling required
- Content reflow at different viewport sizes
- Mobile-first approach ensuring accessibility on all devices

Testing Strategy

Automated Testing:

- Axe-Core accessibility engine integration
- Continuous integration checks for WCAG violations
- Color contrast validation tools
- HTML validation

Manual Testing:

- Screen reader testing (NVDA, JAWS, VoiceOver, TalkBack)
- Keyboard-only navigation verification
- Mobile accessibility testing
- User testing with individuals with disabilities

Open Source Licensing

Recommended License: GNU AGPL-3.0

GNU Affero General Public License v3.0

Rationale

Access to Justice Alignment: The AGPL-3.0 license ensures that all improvements and modifications to the Litigant Portal remain publicly available, directly supporting the access-to-justice mission.

SaaS Protection: Unlike standard GPL, AGPL extends copyleft requirements to network services, preventing organizations from hosting modified versions without sharing improvements.

Community Benefits:

- Courts can freely use and customize the software

- All enhancements must be contributed back to the community
- Prevents proprietary forks that could fragment the ecosystem
- Encourages collaborative development

Court Adoption: Courts benefit from reduced development costs while contributing to shared public infrastructure.

Alternative Consideration

Apache License 2.0 offers a more permissive alternative if court adoption concerns arise:

Advantages:

- More business-friendly terms
- Easier adoption by risk-averse organizations
- Broader license compatibility
- Includes explicit patent grant

Trade-offs:

- No requirement to share modifications
- Potential for proprietary forks
- Less alignment with open access mission

Recommendation: Begin with AGPL-3.0, evaluate based on stakeholder feedback.

Implementation Roadmap

Phase 1: Core Portal MVP (Months 1-3)

Objective: Deploy functional portal to Portsmouth General District Court

Month 1: Foundation

- Django project initialization with selected technology stack
- AWS infrastructure provisioning (ECS, RDS, S3, Cognito)
- Repository pattern implementation
- Core data models (User, Case, Document, Form)
- Authentication integration with AWS Cognito
- CI/CD pipeline configuration

Month 2: Core Features

- User dashboard and profile management
- Document upload and storage workflow
- Static form library for Portsmouth jurisdiction
- Basic case management functionality
- Portsmouth court API integration
- WCAG 2.1 AA compliance implementation

Month 3: eFiling Integration

- Portsmouth-specific eFiling adapter development
- Document submission workflow
- Email and SMS notification system
- User acceptance testing with court staff
- Bug resolution and performance optimization
- Production deployment and monitoring

Key Deliverables:

- Production-ready portal on AWS infrastructure
- Portsmouth users can register, upload documents, and efile
- Functional court system integration
- WCAG 2.1 AA compliant interface
- Complete documentation

Phase 2: Enhanced Workflows (Months 4-6)

Objective: Expand functionality with guided interviews and payment processing

Month 4: Guided Interviews

- A2J Author integration and testing
- HotDocs integration (if required by courts)
- DocAssemble exploration and evaluation
- Form pre-filling from interview answers
- Save and resume functionality
- Multi-step workflow management

Month 5: Financial Processing

- Payment processor abstraction layer
- Stripe integration for card and ACH payments
- Fee waiver application and approval system

- Court fee calculation engine
- Enhanced notification system (deadline reminders)
- SMS integration via AWS SNS
- Templated email system via AWS SES

Month 6: Expansion and Optimization

- Additional Virginia court integrations (2-3 courts)
- Court staff administrative interface
- Analytics and reporting dashboard
- Comprehensive user testing
- Performance optimization and load testing
- Security audit and penetration testing

Key Deliverables:

- Complete guided interview workflow
- Operational payment processing
- Multi-court support (3-5 Virginia courts)
- Enhanced notification capabilities
- Administrative tools for court staff

Phase 3: AI Integration (Months 7-12)

Objective: Deploy AI-powered legal guidance using RAG architecture

Months 7-8: AI Foundation

- AWS Bedrock integration and model evaluation
- OpenSearch vector database configuration
- Legal knowledge base preparation and indexing
- RAG pipeline development (Planner/Worker/Evaluator agents)
- Human-in-the-loop review system
- AI safety guardrails and content filtering

Months 9-10: Legal Assistance Features

- Plain-language legal explanations
- Intelligent case type identification
- Automated form selection assistance
- Document analysis with OCR (AWS Textract)
- Legal terminology glossary with AI-generated definitions
- Jurisdiction-aware response system

Months 11-12: Refinement and Scale

- AI accuracy evaluation and iterative improvement
- Bias detection and mitigation strategies
- Performance optimization for production scale
- Statewide Virginia deployment
- Community feedback collection and integration
- Comprehensive documentation and training materials
- Impact measurement and case studies

Key Deliverables:

- Production-ready AI legal assistant
- RAG system achieving 95%+ accuracy benchmarks
- Deployment to all Virginia General District Courts
- Documented best practices for AI in legal services
- Measurable impact on case outcomes

Technology Stack Evolution Criteria

When to Consider Alternative Technologies

While the recommended Django stack provides a strong foundation, certain scenarios may warrant technology additions or migrations:

Performance Thresholds

Indicator: Response times exceeding 500ms or inability to handle 100+ concurrent users

Recommendation:

- Go microservices for court API integrations (superior concurrency handling)
- Rust for compute-intensive operations (PDF generation, document processing)
- Maintain Django for core business logic

Mobile Application Requirements

Indicator: User research showing 70%+ mobile usage with need for native features (camera, offline access, push notifications)

Recommendation:

- React Native for cross-platform mobile development
- Shared API layer (Django REST Framework or Go)
- Progressive Web App (PWA) as intermediate step

Real-Time Feature Demands

Indicator: Requirements for live case updates, collaborative editing, or real-time communication

Recommendation:

- Go-based WebSocket server for real-time connections
- React for rich client-side state management
- Hybrid architecture maintaining Django for persistence

Complex State Management

Indicator: Guided interviews becoming too complex for HTMX, requiring sophisticated client-side state

Recommendation:

- React components for interactive form builder
- Maintain Django/HTMX for remainder of application
- Progressive enhancement approach

Decision Framework

The architecture's API-first design and repository pattern enable gradual technology migration without complete rewrites. Microservices can be extracted incrementally as specific needs arise.

Migration Path:

1. Extract performance-critical services
2. Maintain stable API contracts
3. Gradual replacement with monitoring
4. Preserve data abstraction layer

Risk Assessment and Mitigation

Technical Risks

Database Performance at Scale

- Risk: PostgreSQL performance degradation with large datasets
- Mitigation: Read replicas, query optimization, caching strategy, connection pooling

Third-Party API Reliability

- Risk: Court system API downtime affecting eFiling
- Mitigation: Retry mechanisms, queue-based architecture, fallback workflows, comprehensive error handling

Security Vulnerabilities

- Risk: Data breaches or unauthorized access
- Mitigation: Regular security audits, automated vulnerability scanning, penetration testing, AWS security best practices

Operational Risks

Court System Integration Complexity

- Risk: Each court having different technical requirements
- Mitigation: Adapter pattern architecture, extensive documentation, court partnership model

User Adoption Challenges

- Risk: Self-represented litigants struggling with technology
- Mitigation: User-centered design, extensive testing, plain language, accessibility focus, support resources

Maintenance and Support

- Risk: Insufficient resources for ongoing maintenance
- Mitigation: Open source community engagement, comprehensive documentation, automated testing, monitoring infrastructure

---

Success Metrics

Technical Metrics

- System uptime: 99.9% availability
- Page load time: Under 2 seconds (median)
- API response time: Under 200ms (95th percentile)

- Error rate: Below 0.1% of requests
- Accessibility: Zero critical WCAG violations

User Metrics

- User registration conversion rate
- Form completion rate
- eFiling success rate
- Time to complete filing (compared to baseline)
- User satisfaction score (CSAT)

Impact Metrics

- Number of self-represented litigants served
- Reduction in procedural errors
- Court staff time savings
- Case processing efficiency improvement
- Geographic and demographic reach

Conclusion

The Litigant Portal technical architecture provides a robust, scalable, and accessible foundation for improving access to justice. The recommended technology stack leverages proven technologies while maintaining flexibility for future enhancements.

Key Strengths

Technical Excellence:

- Modern, maintainable codebase using industry best practices
- Scalable AWS infrastructure supporting growth
- Comprehensive testing and quality assurance

User-Centered Design:

- Mobile-first responsive interface
- WCAG 2.1 AA accessibility compliance
- Plain language and intuitive workflows

Sustainability:

- Open source licensing promoting collaboration
- Modular architecture enabling incremental improvements

- Community-driven internationalization

Mission Alignment:

- Reduces barriers for self-represented litigants
- Eases administrative burden on courts
- Creates reusable public infrastructure

Next Steps

1. Stakeholder review and feedback incorporation
2. Initial project structure creation
3. AWS infrastructure provisioning
4. Development team onboarding
5. Sprint planning for Phase 1

Document Control

Version	Date	Author	Changes
1.0	2025-10-23	Free Law Project	Initial architecture document

Approval

This architecture document requires review and approval from:

- Free Law Project Technical Leadership
- Portsmouth General District Court Representatives
- AWS Partnership Team
- Grant Stakeholders

Contact Information

Jessica Frank, Director of Justice Initiatives, jessica@free.law

Michael Lissner, Executive Director mike@free.law

Free Law Project 6432 Raymond Street Oakland, CA 94609 https://free.law

Granular-Research:

Beware of AI ending up being fake help

I'm currently dealing with TakeCommand 'help' to get registered for our insurance stipend/reimbursement. So far

- Their AI has been very well spoken and provided no real answers
- It's offered to connect me to human help without any kind of reference number, calendar invite - just a 'someone' will reach out to you message and 'sorry i couldnt' help you today'
- They claim to have Insurance Experts or Guidance Counselors to guide you through the process. All links that say 'Schedule a Session' just navigate to a new page or scroll down on the current page to more non-human help - aka links to their online guides and documents that have yet to help me so far
- Their AI help and their online docs reference 'go here' instructions when you login, and those sections on the webpage don't exist - because they have 2 portals, and the AI has no idea which one you're asking about, even though I'm logged into the Hub dashboard when I start the 'help' chat.
- Other help chats that AI fails to help you, has a real person come on at a certain point at least
- I had to call and hold online to get a real person to help, as when I did the 'preserve my place in line' yesterday, the person who called me back put me on hold to 'check some things' for 20+ minutes and never came back
- After calling and waiting on hold, i got an email from the person who was supposed to schedule my help session, asking a quick 'question' to verify my need, that they used the response to tell me what I already know (my current plan is not supported) and sent me back to the online guides and documentation
- This overall is the experience for almost all help phone calls, chats, especially with AI now getting involved - circular 'not my job' type messages that resolve nothing. We must do better!

Native Mobile App development

This is to assess the need for native smartphone features, versus making web code that gets exported to mobile devices and their respective app stores, vs just keeping it a mobile first responsive web app.

- Camera integration for 'document scanning', bar codes, QR codes. First thought here, a court gives QR code to a complex 'help guide' that we've already handled in the Portal, then let's offer to help them through that process more proactively
- Integration to new sharing methods - native phone 'share' mode could give us share endpoints for free as new tech comes out
- Related to 2, any new 'must have' tech that comes out on smart phones will be easier to adopt if we don't have to roll our own integration everytime

- The always mobile web app allows us to control the line between our responsibilities and that of the end user understanding how to use their phone. For example: we make an uploader for supporting documents, but the end user is responsible for knowing how to do a screen grab, download the pdf they need first from another site, maybe fill it out on their phone, save it locally, and find it locally to upload it to our app.

Front End Framework

Weigh and debate the choice of front end frame work. Will most likely be driven by the decision for Mobile App native features as listed above. The main pro/con argument is the major benefit of using our current tech stack experience in django/python vs the needs of the Portal and the Litigan Portal future development.

Django Cotton, MDX

- Takes advantage of current tech stack expertise
- Allows for redundancy on support and maintenance as no single dev/skillset holds the keys to the castle

React

- Massive developer pool, tooling, and acceptance
- Helps for open source contributions
- Allows for easier switching of related tech (css, build tools, browser testing, browser features)
- Write pure typescript, JSX, web components - no wrappers or mods to existing tools to add JSX, web components, and styles. In a pinch can always just write some JS, HMTL, and CSS to solve a problem
- Still a web app on mobile

React Native

- Access to native phone interfaces and APIs - camera, share, speed
- Build once, deploy everywhere - mobile native, web, etc
- Adds another tool to our kit that won’t be going away any time soon. FLP Mobile division is born

Other - Flutter(Dart)/Kotlin/Iconic

- I list these here because they are out there and being used in this space, with various advantages and disadvantages, but still worth at least a cursory look.